Prerequisite Mathematical Definitions

1 Transition Independence

Setting. We consider functions $f\colon S_1 \times \cdots \times S_n \to T$, where the factor sets S_i and the codomain T carry no assumed structure unless otherwise noted. We write $\boldsymbol{a}|_{a_i=a_i'}$ for the tuple identical to $\boldsymbol{a}$ except that the i-th component is replaced by a_i', and write

$$\mathcal{D}_i \coloneqq \{(f(\boldsymbol{a}),\, a_i) : \boldsymbol{a} \in S_1 \times \cdots \times S_n\} \subseteq T \times S_i$$

for the joint range of (f, pr_i) — the set of (v, a_i) pairs realized by some $\boldsymbol{a}$. The auxiliary functions introduced in the conditions below are defined on $\mathcal{D}_i$ (or $\mathcal{D}_i \times S_i$).

We introduce three formulations of transition independence forming a hierarchy of decreasing generality: the *transition-map* condition requires no structure on T; the *finite-difference* condition requires that $(T, +)$ be an abelian group, so that subtraction is defined; and the *derivative* condition requires that T be a normed vector space, f be differentiable in s_i, and suitable regularity conditions hold. Each formulation is stated at its natural level of generality.

Transition map. The variable s_i is *transition-independent* of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f iff there exists a *transition map* $H_i\colon \mathcal{D}_i \times S_i \to T$ (necessarily unique on its declared domain) such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, a_i' \in S_i \quad f(\boldsymbol{a}|_{a_i=a_i'}) = H_i\big(f(\boldsymbol{a}),\, a_i,\, a_i'\big). \tag{1}$$

The non-trivial content of the condition is that the remaining coordinates $\boldsymbol{a}_{j\neq i}$ are dropped in favor of the output $f(\boldsymbol{a})$. No structure on T is required: the condition is meaningful for functions valued in discrete spaces, ordered sets, manifolds, or any other codomain. For H_i to be well-defined as a function on $\mathcal{D}_i \times S_i$, the following consistency condition is necessary and sufficient.

Remark 1 (Output-determinacy). For every $\boldsymbol{a}, \boldsymbol{b} \in S_1 \times \cdots \times S_n$, if $f(\boldsymbol{a}) = f(\boldsymbol{b})$ and $a_i = b_i$, then $f(\boldsymbol{a}|_{a_i=a_i'}) = f(\boldsymbol{b}|_{b_i=a_i'})$ for every $a_i' \in S_i$.

Proof of the equivalence. ($\Rightarrow$) If H_i satisfies (1) and $f(\boldsymbol{a}) = f(\boldsymbol{b})$ with $a_i = b_i$, then $f(\boldsymbol{a}|_{a_i=a_i'}) = H_i(f(\boldsymbol{a}), a_i, a_i') = H_i(f(\boldsymbol{b}), b_i, a_i') = f(\boldsymbol{b}|_{b_i=a_i'})$. ($\Leftarrow$) Given output-determinacy, define $H_i(v, s, a_i') \coloneqq f(\boldsymbol{a}|_{a_i=a_i'})$ for any $\boldsymbol{a}$ with $f(\boldsymbol{a}) = v$ and $a_i = s$; output-determinacy makes the choice of $\boldsymbol{a}$ immaterial, so H_i is well-defined and (1) holds by construction.

Geometrically, output-determinacy says that the level sets of f do not "twist" along the s_i-direction: two points sharing both the same output and the same i-th coordinate must land on the same level set as each other — in general a different one from where they started — when s_i is varied. In full generality, the condition asserts that the pair $(f(\boldsymbol{a}), a_i)$ is a sufficient representation for predicting how f responds to movement of s_i.

Finite-difference formulation. When $(T, +)$ is an abelian group, the transition-map condition has an equivalent algebraic formulation: there exists $G_i\colon \mathcal{D}_i \times S_i \to T$ such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, a_i' \in S_i \quad f(\boldsymbol{a}|_{a_i=a_i'}) - f(\boldsymbol{a}) = G_i\big(f(\boldsymbol{a}),\, a_i,\, a_i'\big). \tag{2}$$

The two formulations are related by $G_i(v, a_i, a_i') = H_i(v, a_i, a_i') - v$ and $H_i(v, a_i, a_i') = v + G_i(v, a_i, a_i')$.

Derivative formulation. Suppose S_i is an open interval of $\mathbb{R}$ and T is a normed vector space. When f is differentiable in s_i, under appropriate regularity conditions, the derivative condition is: there exists $F_i\colon \mathcal{D}_i \to T$ such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n \quad \left.\frac{\partial f(\boldsymbol{s})}{\partial s_i}\right|_{\boldsymbol{s}=\boldsymbol{a}} = F_i\big(f(\boldsymbol{a}),\, a_i\big). \tag{3}$$

When (3) holds, the map $t \mapsto f(\boldsymbol{a}|_{a_i=t})$ satisfies the initial value problem

$$\frac{dy}{dt} = F_i(y, t), \qquad y(a_i) = f(\boldsymbol{a}), \tag{4}$$

whose solution determines $f(\boldsymbol{a}|_{a_i=a_i'})$ entirely from $(f(\boldsymbol{a}), a_i)$ provided the solution is unique. The ODE (4) is the bridge between the derivative condition and the transition map.

Summary of equivalences. The relationships among (1), (2), and (3) can be summarized as follows:

(a) Condition (1) is equivalent to output-determinacy (Remark 1). No structure on T is required.

(b) If $(T, +)$ is an abelian group, conditions (1) and (2) are equivalent via $H_i(v, a_i, a_i') = v + G_i(v, a_i, a_i')$ and $G_i(v, a_i, a_i') = H_i(v, a_i, a_i') - v$.

(c) Suppose S_i is an open interval of $\mathbb{R}$, T is a normed vector space, and f is differentiable in s_i throughout. Then (1) $\Rightarrow$ (3) holds, via $F_i(v, a_i) := \partial_{a_i'} H_i(v, a_i, a_i')\big|_{a_i'=a_i}$. The reverse direction (3) $\Rightarrow$ (1) holds provided the IVP (4) admits a unique solution from each initial datum $(v, a_i) \in \mathcal{D}_i$. Under these hypotheses all three formulations are equivalent. The trajectory $t \mapsto f(\boldsymbol{a}|_{a_i=t})$ solves the initial value problem

$$\frac{dy}{dt} = F_i(y, t), \qquad y(a_i) = f(\boldsymbol{a}),$$

and, provided the integrand below is integrable — for instance when f is continuously differentiable in s_i — the finite-difference and derivative formulations are related by

$$G_i\big(f(\boldsymbol{a}), a_i, a_i'\big) = \int_{a_i}^{a_i'} F_i\big(f(\boldsymbol{a}|_{a_i=t}),\, t\big)\, dt. \tag{5}$$

2 Sufficient Context and Context Bases

Setting. Transition independence may be partial: s_i might be transition-independent of some variables, with the others retained as its *context*, without being fully independent. We formalize this by partitioning $\{1,\dots,n\}$ as $\{i\} \cup J \cup K$, where J is the context and $K = \{1,\dots,n\} \setminus (\{i\} \cup J)$ contains the remaining variables. The variable s_i is *transition-independent of $\boldsymbol{s}_K$ given context $\boldsymbol{s}_J$* if the equations of Section 1 hold when the auxiliary maps $H_{i,J}$, $G_{i,J}$, or $F_{i,J}$ are allowed to depend on the context values $\boldsymbol{a}_J$. Each of the three formulations of Section 1 extends to this conditional setting.

Transition-map formulation (no structure on T). The variable s_i is transition-independent of $\boldsymbol{s}_K$ given context $\boldsymbol{s}_J$ iff there exists a map $H_{i,J}\colon \mathcal{D}_i^J \times S_i \to T$ (denoted $H_{i,J}(f(\boldsymbol{a}), a_i, a_i', \boldsymbol{a}_J)$, with the target a_i' written before the context $\boldsymbol{a}_J$) such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, a_i' \in S_i \quad f(\boldsymbol{a}|_{a_i=a_i'}) = H_{i,J}\big(f(\boldsymbol{a}),\, a_i,\, a_i',\, \boldsymbol{a}_J\big), \tag{6}$$

where $\mathcal{D}_i^J \coloneqq \{(f(\boldsymbol{a}),\, a_i,\, \boldsymbol{a}_J) : \boldsymbol{a} \in S_1 \times \cdots \times S_n\}$. Setting $J = \emptyset$ recovers (1); setting $J = \{1,\dots,n\} \setminus \{i\}$ holds trivially.

Remark 2 (Output-determinacy on context J). For every $\boldsymbol{a}, \boldsymbol{b} \in S_1 \times \cdots \times S_n$, if $f(\boldsymbol{a}) = f(\boldsymbol{b})$, $a_i = b_i$, and $\boldsymbol{a}_J = \boldsymbol{b}_J$, then $f(\boldsymbol{a}|_{a_i=a_i'}) = f(\boldsymbol{b}|_{b_i=a_i'})$ for every $a_i' \in S_i$.

As in the full case, output-determinacy on context J is necessary and sufficient for $H_{i,J}$ to be well-defined on $\mathcal{D}_i^J \times S_i$. Setting $J = \emptyset$ recovers Remark 1.

Geometrically, this is still essentially the no-twist condition of Section 1, now imposed within each context slice: with $\boldsymbol{s}_J$ fixed, the level sets of f do not "twist" along the s_i-direction.

Remark 3 (Partial-function form). For every $\boldsymbol{c}_J \in \prod_{j\in J} S_j$, the variable s_i is fully transition-independent on the partial function $f|_{\boldsymbol{s}_J=\boldsymbol{c}_J}\colon \prod_{m\in\{i\}\cup K} S_m \to T$, in the sense of Section 1.

Remark 3 is equivalent to s_i being transition-independent of $\boldsymbol{s}_K$ given context $\boldsymbol{s}_J$: since $\boldsymbol{a}|_{a_i=a_i'}$ fixes $\boldsymbol{a}_J$, the map $H_{i,J}(\,\cdot\,, \boldsymbol{a}_J)$ is the transition map of $f|_{\boldsymbol{s}_J=\boldsymbol{c}_J}$.

In other words, partial transition independence is full transition independence on sets of partial functions. Context is what's fixed.

Finite-difference formulation (abelian T). When $(T, +)$ is an abelian group, the condition has the equivalent algebraic form: there exists $G_{i,J}\colon \mathcal{D}_i^J \times S_i \to T$ such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, a_i' \in S_i \quad f(\boldsymbol{a}|_{a_i=a_i'}) - f(\boldsymbol{a}) = G_{i,J}\big(f(\boldsymbol{a}),\, a_i,\, a_i',\, \boldsymbol{a}_J\big). \tag{7}$$

The two formulations are related by $G_{i,J}(v, a_i, a_i', \boldsymbol{a}_J) = H_{i,J}(v, a_i, a_i', \boldsymbol{a}_J) - v$ and $H_{i,J}(v, a_i, a_i', \boldsymbol{a}_J) = v + G_{i,J}(v, a_i, a_i', \boldsymbol{a}_J)$.

Derivative formulation (S_i real, T normed). Suppose S_i is an open interval of $\mathbb{R}$ and T is a normed vector space. When f is differentiable in s_i, under appropriate regularity conditions, the condition becomes: there exists $F_{i,J}\colon \mathcal{D}_i^J \to T$ such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n \quad \left.\frac{\partial f(\boldsymbol{s})}{\partial s_i}\right|_{\boldsymbol{s}=\boldsymbol{a}} = F_{i,J}\big(f(\boldsymbol{a}),\, a_i,\, \boldsymbol{a}_J\big). \tag{8}$$

When (8) holds, the map $t \mapsto f(\boldsymbol{a}|_{a_i=t})$ satisfies the initial value problem

$$\frac{dy}{dt} = F_{i,J}(y, t, \boldsymbol{a}_J), \qquad y(a_i) = f(\boldsymbol{a}), \tag{9}$$

whose solution determines $f(\boldsymbol{a}|_{a_i=a_i'})$ entirely from $(f(\boldsymbol{a}), a_i, \boldsymbol{a}_J)$ provided the solution is unique. The ODE (9) is the bridge between the derivative condition and the transition map.

Summary of equivalences. The relationships among (6), (7), and (8) parallel those of Section 1, with the context tuple $\boldsymbol{a}_J$ entering each auxiliary map as a passenger argument:

(a) Condition (6) is equivalent to output-determinacy on context J (Remark 2). No structure on T is required.

(b) If $(T, +)$ is an abelian group, conditions (6) and (7) are equivalent via $H_{i,J}(v, a_i, a_i', \boldsymbol{a}_J) = v + G_{i,J}(v, a_i, a_i', \boldsymbol{a}_J)$ and $G_{i,J}(v, a_i, a_i', \boldsymbol{a}_J) = H_{i,J}(v, a_i, a_i', \boldsymbol{a}_J) - v$.

(c) Suppose S_i is an open interval of $\mathbb{R}$, T is a normed vector space, and f is differentiable in s_i throughout. Then (6) $\Rightarrow$ (8) holds. The reverse direction (8) $\Rightarrow$ (6) holds provided the IVP (9) admits a unique solution from each initial datum. Under these hypotheses all three formulations are equivalent. The trajectory $t \mapsto f(\boldsymbol{a}|_{a_i=t})$ solves the initial value problem

$$\frac{dy}{dt} = F_{i,J}(y, t, \boldsymbol{a}_J), \qquad y(a_i) = f(\boldsymbol{a}),$$

and, provided the integrand below is integrable — for instance when f is continuously differentiable in s_i — the finite-difference and derivative formulations are related by

$$G_{i,J}\big(f(\boldsymbol{a}), a_i, a_i', \boldsymbol{a}_J\big) = \int_{a_i}^{a_i'} F_{i,J}\big(f(\boldsymbol{a}|_{a_i=t}),\, t,\, \boldsymbol{a}_J\big)\, dt. \tag{10}$$

From now on we work with the transition-map version (6).

3 Transform Independence

Setting. Sections 1 and 2 describe the change in a variable by its initial and final values. This section describes it by a transform carrying the initial value to the final one. Fix a coordinate i, and let Δ_i be a *family* of *transforms* of S_i: each $\delta_i \in \Delta_i$ is a map $\delta_i \colon S_i \to S_i$, and we write $\delta_i * a_i$ for $\delta_i(a_i)$, so that the substitution of a_i' for a_i is *represented* by a transform δ_i if $a_i' = \delta_i * a_i$. No structure on the family is assumed; in particular Δ_i need not be closed under composition. We write

$$\mathcal{E}_i \coloneqq \{(f(\boldsymbol{a}),\, \delta_i) : \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \delta_i \in \Delta_i\} = f(S_1 \times \cdots \times S_n) \times \Delta_i$$

for the product of the range of f with the family — the value and the transform do not constrain each other, since δ_i is chosen independently of $\boldsymbol{a}$. The auxiliary functions introduced in the conditions below are defined on $\mathcal{E}_i$: they consult the transform applied, not the initial value a_i or the final value a_i'.

Transform-map formulation (no structure on T). The variable s_i is *transform-independent* of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i iff there exists a *transform map* $H_i^{\Delta} \colon \mathcal{E}_i \to T$ (necessarily unique on its declared domain) such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, \delta_i \in \Delta_i \quad f(\boldsymbol{a}|_{a_i = \delta_i * a_i}) = H_i^{\Delta}\big(f(\boldsymbol{a}),\, \delta_i\big). \tag{11}$$

The non-trivial content of the condition is that the remaining coordinates $\boldsymbol{a}_{j\neq i}$ and the initial value a_i are both dropped in favor of the output $f(\boldsymbol{a})$ and the transform δ_i alone. For H_i^{Δ} to be well-defined as a function on $\mathcal{E}_i$, the following consistency condition is necessary and sufficient.

Remark 4 (Transform output-determinacy). For every $\boldsymbol{a}, \boldsymbol{b} \in S_1 \times \cdots \times S_n$, if $f(\boldsymbol{a}) = f(\boldsymbol{b})$, then $f(\boldsymbol{a}|_{a_i = \delta_i * a_i}) = f(\boldsymbol{b}|_{b_i = \delta_i * b_i})$ for every $\delta_i \in \Delta_i$.

Proof of the equivalence. ($\Rightarrow$) If H_i^{Δ} satisfies (11) and $f(\boldsymbol{a}) = f(\boldsymbol{b})$, then $f(\boldsymbol{a}|_{a_i = \delta_i * a_i}) = H_i^{\Delta}(f(\boldsymbol{a}), \delta_i) = H_i^{\Delta}(f(\boldsymbol{b}), \delta_i) = f(\boldsymbol{b}|_{b_i = \delta_i * b_i})$ for every $\delta_i \in \Delta_i$. ($\Leftarrow$) Given transform output-determinacy, define $H_i^{\Delta}(v, \delta_i) \coloneqq f(\boldsymbol{a}|_{a_i = \delta_i * a_i})$ for any $\boldsymbol{a}$ with $f(\boldsymbol{a}) = v$; the condition makes the choice of $\boldsymbol{a}$ immaterial, so H_i^{Δ} is well-defined and (11) holds by construction.

Transform output-determinacy says that two tuples sharing the same output must respond identically to each other under every transform in Δ_i, whatever their initial values. Over the substitutions that Δ_i represents, ordinary output-determinacy (Remark 1) makes that same demand, but only of tuples that also share an initial value: for those, the transform applied and the final value describe the same substitution. Geometrically, the condition says that applying a transform δ_i to the i-th coordinate carries every level set of f into a single level set — in general a different one — and so descends along f to the induced map $v \mapsto H_i^{\Delta}(v, \delta_i)$ on the range of f. In full generality, the condition asserts that the output $f(\boldsymbol{a})$ alone is a sufficient representation for predicting how f responds to each transform in Δ_i.

Finite-difference formulation (abelian T). When $(T, +)$ is an abelian group, the transform-map condition has an equivalent algebraic formulation: there exists $G_i^{\Delta} \colon \mathcal{E}_i \to T$ such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, \delta_i \in \Delta_i \quad f(\boldsymbol{a}|_{a_i = \delta_i * a_i}) - f(\boldsymbol{a}) = G_i^{\Delta}\big(f(\boldsymbol{a}),\, \delta_i\big). \tag{12}$$

The two formulations are related by $G_i^{\Delta}(v, \delta_i) = H_i^{\Delta}(v, \delta_i) - v$ and $H_i^{\Delta}(v, \delta_i) = v + G_i^{\Delta}(v, \delta_i)$. No derivative (instantaneous) formulation is included; the two formulations above are the only ones we use.

Summary of equivalences. The relationships between (11) and (12) parallel items (a) and (b) of Section 1, with the pair $(f(\boldsymbol{a}),\, \delta_i)$ replacing $(f(\boldsymbol{a}),\, a_i,\, a_i')$ throughout:

(a) Condition (11) is equivalent to transform output-determinacy (Remark 4). No structure on T is required.

(b) If $(T, +)$ is an abelian group, conditions (11) and (12) are equivalent via $H_i^{\Delta}(v, \delta_i) = v + G_i^{\Delta}(v, \delta_i)$ and $G_i^{\Delta}(v, \delta_i) = H_i^{\Delta}(v, \delta_i) - v$.

Remark 5 (Keeping the initial value). Passing from the transition-map condition (1) to the transform-map condition (11) replaces the pair (a_i, a_i') with the transform δ_i alone. If we replaced it instead with the pair (a_i, δ_i) — the initial value kept — the condition would read: there exists $H\colon \mathcal{D}_i \times \Delta_i \to T$ (necessarily unique on its declared domain) such that

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, \delta_i \in \Delta_i \quad f(\boldsymbol{a}|_{a_i = \delta_i * a_i}) = H\big(f(\boldsymbol{a}),\, a_i,\, \delta_i\big). \tag{13}$$

Condition (13) is equivalent to condition (1) quantified over the substitutions that Δ_i represents: there exists H_i such that $f(\boldsymbol{a}|_{a_i = a_i'}) = H_i(f(\boldsymbol{a}), a_i, a_i')$ for every $\boldsymbol{a} \in S_1 \times \cdots \times S_n$ and every target $a_i' = \delta_i * a_i$ with $\delta_i \in \Delta_i$.

Proof of the equivalence. ($\Rightarrow$) Given H satisfying (13), define $H_i(v, a_i, a_i') := H(v, a_i, \delta_i)$ for any $\delta_i \in \Delta_i$ with $\delta_i * a_i = a_i'$. The choice is immaterial: for any $\boldsymbol{a}$ realizing (v, a_i), every representing δ_i gives $H(v, a_i, \delta_i) = f(\boldsymbol{a}|_{a_i = \delta_i * a_i}) = f(\boldsymbol{a}|_{a_i = a_i'})$ — the substituted tuple depends only on the final value, not on the transform that represents the substitution — so H_i is well-defined and the restricted condition holds by construction. ($\Leftarrow$) Given H_i, set $H(v, a_i, \delta_i) := H_i(v, a_i, \delta_i * a_i)$; the target $\delta_i * a_i$ is represented by δ_i itself, so (13) holds. When every substitution of a_i' for a_i is represented by some transform in Δ_i, the restriction disappears: condition (13) is then equivalent to condition (1) itself.

Remark 6 (Reach and strictness). Deleting the initial value from the pair (a_i, δ_i) of the preceding remark removes information and nothing else: any transform map serves, through the preceding remark, as a transition map on the substitutions that Δ_i represents. Transform output-determinacy (Remark 4) therefore implies output-determinacy (Remark 1) over the substitutions that Δ_i represents. The implication reaches further. We write $\overline{\Delta}_i$ for the family of maps of S_i carried out by finitely many transforms in Δ_i applied in succession, each to the value the previous one produced; a single transform counts as a succession of length one. Each map in $\overline{\Delta}_i$ is itself a transform of S_i, so the transform-map condition (11) applies with $\overline{\Delta}_i$ in the role of Δ_i.

Lemma 7 (Passage to the closure). *If s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i, then it is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and $\overline{\Delta}_i$: there exists a transform map $H_i^{\overline{\Delta}}\colon f(S_1\times\cdots\times S_n)\times\overline{\Delta}_i \to T$ (necessarily unique on its declared domain) such that*

$$\forall\, \boldsymbol{a} \in S_1 \times \cdots \times S_n,\ \ \forall\, \overline{\delta}_i \in \overline{\Delta}_i \quad f(\boldsymbol{a}|_{a_i = \overline{\delta}_i * a_i}) = H_i^{\overline{\Delta}}\big(f(\boldsymbol{a}),\, \overline{\delta}_i\big). \tag{14}$$

Proof. Suppose H_i^{Δ} satisfies (11). For a tuple $\boldsymbol{a}$ and a succession $\delta_{i,1}, \ldots, \delta_{i,k} \in \Delta_i$, write $a_i^0 := a_i$ and $a_i^j := \delta_{i,j} * a_i^{j-1}$ for the values the succession visits, so that a succession carrying out $\overline{\delta}_i \in \overline{\Delta}_i$ ends at $a_i^k = \overline{\delta}_i * a_i$. The tuple $\boldsymbol{a}|_{a_i = a_i^{j-1}}$ lies in $S_1 \times \cdots \times S_n$ and carries initial value a_i^{j-1}, so (11) applied to it with $\delta_{i,j}$ gives $f(\boldsymbol{a}|_{a_i = a_i^j}) = H_i^{\Delta}\big(f(\boldsymbol{a}|_{a_i = a_i^{j-1}}),\, \delta_{i,j}\big)$: each response is again an output of f and feeds the next application, and induction on j gives $f(\boldsymbol{a}|_{a_i = a_i^k}) = H_i^{\Delta}(\cdots H_i^{\Delta}(f(\boldsymbol{a}),\, \delta_{i,1}) \cdots ,\, \delta_{i,k})$ — a value determined by the output $f(\boldsymbol{a})$ and the succession alone. Now define $H_i^{\overline{\Delta}}(v, \overline{\delta}_i)$ by chaining H_i^{Δ} from v along any succession carrying out $\overline{\delta}_i$. The choice is immaterial: every v in the declared domain is realized as $v = f(\boldsymbol{a})$ for some $\boldsymbol{a}$, and two successions carrying out the same map substitute the same value into $\boldsymbol{a}$, so both chains compute the output of the same substituted tuple, $f(\boldsymbol{a}|_{a_i = \overline{\delta}_i * a_i})$. Hence $H_i^{\overline{\Delta}}$ is well-defined on its declared domain, and (14) holds by construction.

Proposition 8 (Reach). *Suppose $\overline{\Delta}_i$ represents every substitution of one value for another: for every $a_i, a_i' \in S_i$ with $a_i' \neq a_i$, some $\overline{\delta}_i \in \overline{\Delta}_i$ has $a_i' = \overline{\delta}_i * a_i$. If s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i, then s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f.*

Proof. Lemma 7 yields $H_i^{\overline{\Delta}}$ satisfying (14). Through Remark 5, with $\overline{\Delta}_i$ in the role of Δ_i, it serves as a transition map on the substitutions that $\overline{\Delta}_i$ represents: there exists H_i with $f(\boldsymbol{a}|_{a_i = a_i'}) = H_i\big(f(\boldsymbol{a}),\, a_i,\, a_i'\big)$ for every $\boldsymbol{a}$ and every target $a_i' = \overline{\delta}_i * a_i$, $\overline{\delta}_i \in \overline{\Delta}_i$. By hypothesis every target $a_i' \neq a_i$ is of that form; the identity target is free: the substituted tuple is $\boldsymbol{a}$ itself, so setting $H_i(v, a_i, a_i) := v$ satisfies (1) there and agrees with any value already forced. Thus H_i is defined on all of $\mathcal{D}_i \times S_i$ and (1) holds: s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f, and output-determinacy (Remark 1) follows by item (a) of Section 1.

Corollary 9 (Represented substitutions)**.** *If s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i, then there exists H_i such that $f(\boldsymbol{a}|_{a_i=a_i'}) = H_i(f(\boldsymbol{a}), a_i, a_i')$ for every $\boldsymbol{a} \in S_1 \times \cdots \times S_n$ and every target $a_i' = \overline{\delta}_i * a_i$ with $\overline{\delta}_i \in \overline{\Delta}_i$ — condition (1) quantified over the substitutions that $\overline{\Delta}_i$ represents.*

Proof. Lemma 7 yields $H_i^{\overline{\Delta}}$ satisfying (14). Through Remark 5, with $\overline{\Delta}_i$ in the role of Δ_i, it serves as a transition map on the substitutions that $\overline{\Delta}_i$ represents.
Transform independence alone yields the map: no hypothesis on the family enters.

Definition 10 (Achievability)**.** A value $a_i' \in S_i$ is *achievable* from $a_i \in S_i$ with respect to Δ_i iff $a_i' = a_i$ or some $\overline{\delta}_i \in \overline{\Delta}_i$ has $a_i' = \overline{\delta}_i * a_i$: either finitely many transforms in Δ_i applied in succession carry a_i to a_i', or a_i' is a_i itself. The *achievable set* of a_i is the set of values achievable from a_i.

Achievable sets are closed under the transforms: appending one further transform extends a succession. A subset $R \subseteq S_i$ is the achievable set of each of its values iff R is closed under the transforms and every value of R is achievable from every other. The achievable set of a single value meets the first requirement but need not meet the second: a succession may carry a_i to a_i' while no succession carries a_i' back to a_i. In this vocabulary, the targets of Corollary 9, together with a_i itself, are exactly the values achievable from a_i.

Corollary 11 (Achievable sets)**.** *Suppose $R \subseteq S_i$ is the achievable set of each of its values. If s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i, then s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to the partial function $f|_{s_i\in R}\colon S_1 \times \cdots \times S_{i-1} \times R \times S_{i+1} \times \cdots \times S_n \to T$ with the i-th factor confined to R, in the sense of Section 1.*

Proof. The equivalence above unpacks the hypothesis: R is closed under the transforms, and every value of R is achievable from every other. Each $\delta_i \in \Delta_i$ carries R into R; write $\delta_i|_R$ for the transform of R it induces, and $\Delta_i|_R$ for the family of these. The partial function agrees with f at every tuple of its domain; by (11), any two such tuples sharing an output respond identically to each other under every $\delta_i \in \Delta_i$, and each response is again a value of the partial function: the substituted value $\delta_i|_R * a_i = \delta_i * a_i$ lies in R. Hence transform output-determinacy (Remark 4) holds for $f|_{s_i\in R}$ and $\Delta_i|_R$, and by item (a) of Section 3 s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to $f|_{s_i\in R}$ and $\Delta_i|_R$. A succession in Δ_i starting at a value of R visits only values of R, and the induced succession in $\Delta_i|_R$ carries out the same map of R; since every value of R is achievable from every other, $\overline{\Delta_i|_R}$ represents every substitution of one value of R for another. Proposition 8, with $f|_{s_i\in R}$ in the role of f, R in the role of S_i, and $\Delta_i|_R$ in the role of Δ_i, yields the claim: s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to $f|_{s_i\in R}$.
Setting $R = S_i$ recovers Proposition 8: closedness holds trivially, and $\overline{\Delta}_i$ represents every substitution of one value for another iff every value of S_i is achievable from every other.

Remark 12 (Safe transforms). Transform independence with respect to a family whose closure represents every substitution of one value for another gives, through Proposition 8, transition independence. Whether some family of transforms of S_i satisfies both of those hypotheses is decided by f alone. Call a transform δ_i of S_i *safe* for f when every two tuples sharing the same output but not the same initial value respond identically to each other under δ_i: if $f(\boldsymbol{a}) = f(\boldsymbol{b})$ and $a_i \neq b_i$, then $f(\boldsymbol{a}|_{a_i=\delta_i * a_i}) = f(\boldsymbol{b}|_{b_i=\delta_i * b_i})$. We write Δ_i^f for the family of all safe transforms of S_i. The identity transform of S_i is safe for every f: the substituted tuples are $\boldsymbol{a}$ and $\boldsymbol{b}$ themselves.

Lemma 13 (Closure of the safe family)**.** *Suppose s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f. Then every map of S_i carried out by finitely many safe transforms applied in succession, each to the value the previous one produced, is itself safe for f. With Δ_i^f in the role of Δ_i in Remark 6, the closure $\overline{\Delta}_i$ is Δ_i^f itself.*

Proof. For $\boldsymbol{a}, \boldsymbol{b}$ with $f(\boldsymbol{a}) = f(\boldsymbol{b})$ and $a_i \neq b_i$, and a succession $\delta_{i,1}, \ldots, \delta_{i,k}$ of safe transforms carrying out $\overline{\delta}_i$, write $a_i^0 := a_i$ and $a_i^j := \delta_{i,j} * a_i^{j-1}$ for the values the succession visits from a_i, and likewise b_i^j from b_i, so that $a_i^k = \overline{\delta}_i * a_i$ and $b_i^k = \overline{\delta}_i * b_i$. The substituted tuples $\boldsymbol{a}|_{a_i=a_i^j}$ and $\boldsymbol{b}|_{b_i=b_i^j}$ lie in $S_1 \times \cdots \times S_n$, carry initial values a_i^j and b_i^j, and share an output at $j = 0$; sharing survives each step. If $a_i^{j-1} = b_i^{j-1}$, the tuples at $j-1$ share their initial value, and output-determinacy (Remark 1), which holds by item (a) of Section 1, keeps the outputs equal at the common target $a_i^j = b_i^j$. If $a_i^{j-1} \neq b_i^{j-1}$, the tuples at $j-1$ share the same output but not the same initial value, and $\delta_{i,j}$ is safe. Induction on j gives $f(\boldsymbol{a}|_{a_i=\overline{\delta}_i * a_i}) = f(\boldsymbol{b}|_{b_i=\overline{\delta}_i * b_i})$: the map the succession carries out is safe. The closure clause follows: every map in $\overline{\Delta}_i$ is carried out by a succession of safe transforms, and every safe transform lies in $\overline{\Delta}_i$ as a succession of length one.

Lemma 14 (Families of safe transforms)**.** *Suppose s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f. Then s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i iff every member of Δ_i is safe for f.*

Proof. ($\Rightarrow$) If H_i^Δ satisfies (11), then for $f(\boldsymbol{a}) = f(\boldsymbol{b})$ with $a_i \neq b_i$ and any $\delta_i \in \Delta_i$, $f(\boldsymbol{a}|_{a_i=\delta_i * a_i}) = H_i^\Delta(f(\boldsymbol{a}), \delta_i) = H_i^\Delta(f(\boldsymbol{b}), \delta_i) = f(\boldsymbol{b}|_{b_i=\delta_i * b_i})$: every member is safe. ($\Leftarrow$) Suppose every member of Δ_i is safe, and let $f(\boldsymbol{a}) = f(\boldsymbol{b})$ with $\delta_i \in \Delta_i$. If $a_i = b_i$, then $\delta_i * a_i = \delta_i * b_i$, and output-determinacy (Remark 1), which holds by item (a) of Section 1, keeps the outputs equal at that common target; if $a_i \neq b_i$, δ_i is safe. Transform output-determinacy (Remark 4) therefore holds, and s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i by item (a) of Section 3.

Proposition 15 (Reach of the safe family)**.** *Suppose s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f. Then s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and some family Δ_i of transforms of S_i satisfying the hypothesis of Proposition 8 — its closure $\overline{\Delta}_i$ represents every substitution of one value for another — iff the safe family Δ_i^f itself represents every substitution of one value for another. In that case Δ_i^f is such a family.*

Proof. ($\Rightarrow$) Let Δ_i be such a family: every member is safe (Lemma 14), and every map in $\overline{\Delta}_i$ is carried out by finitely many members applied in succession, hence is itself safe (Lemma 13). Every substitution of one value for another is represented by some map in $\overline{\Delta}_i$, hence by a safe transform: Δ_i^f represents every substitution of one value for another. ($\Leftarrow$) Take Δ_i^f in the role of Δ_i — this also proves the closing assertion. Every member of Δ_i^f is safe, so s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i^f (Lemma 14); the closure of Δ_i^f is Δ_i^f itself (Lemma 13), and it represents every substitution of one value for another by hypothesis.

Corollary 16 (Strictness)**.** *Suppose s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f, and let Δ_i be a family of transforms of S_i satisfying the hypothesis of Proposition 8. The implication of Proposition 8 is strict at f and Δ_i — s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f, yet not transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and Δ_i — iff some member of Δ_i is not safe for f.*

Proof. Lemma 14: transform independence with respect to f and Δ_i holds iff every member of Δ_i is safe for f; strictness at f and Δ_i is exactly its failure.

Remark 17 (Assignments). For each value $c \in S_i$, the *assignment* to c is the transform of S_i carrying every value to c. The assignment family represents every substitution of one value for another (the substitution of a_i' for a_i is represented by the assignment to a_i'), and a succession of assignments carries out its last member, so the family is its own closure. Suppose s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f. By Lemma 14, s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and the assignment family iff every assignment is safe for f — iff $f(\boldsymbol{a}|_{a_i=c})$ is determined by $(f(\boldsymbol{a}), c)$ alone. If every assignment is safe, the assignment family lies inside Δ_i^f, and Δ_i^f then represents every substitution of one value for another; the reverse implication need not hold.

Remark 18 (Determination of the initial value). Suppose s_i is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ with respect to f, and suppose the output determines the initial value: $f(\boldsymbol{a}) = f(\boldsymbol{b})$ only when $a_i = b_i$. Then two tuples sharing the same output share the same initial value, every transform of S_i is safe for f, and s_i is transform-independent of the remaining variables $\boldsymbol{s}_{j\neq i}$ and of its own initial value with respect to f and every family Δ_i of transforms of S_i (Lemma 14), whether or not its closure represents every substitution of one value for another.

Remark 19 (Equivariance form). For $\delta_i \in \Delta_i$, we write $\hat{\delta}_i$ for the map $\boldsymbol{a} \mapsto \boldsymbol{a}|_{a_i=\delta_i * a_i}$ on $S_1 \times \cdots \times S_n$, which applies δ_i to the i-th coordinate and leaves the remaining coordinates unchanged. Condition (11) is then the intertwining identity

$$f \circ \hat{\delta}_i = \rho(\delta_i) \circ f, \qquad \rho(\delta_i) \coloneqq H_i^{\Delta}(\,\cdot\,, \delta_i).$$

Equivariance in the usual sense supplies actions on both sides and asks f to intertwine them; here only the domain side is given, and the codomain family ρ is quantified existentially. Transform output-determinacy (Remark 4) says exactly that the level sets of f form a congruence for each $\hat{\delta}_i$: each level set is carried into a single level set, in general a different one. Each $\hat{\delta}_i$ therefore descends along f to $\rho(\delta_i)$, a self-map of the range of f, unique there because its values are forced. Composites descend as well. Since $\widehat{\delta_i' \circ \delta_i} = \hat{\delta}_i' \circ \hat{\delta}_i$ and the induced maps are forced, $\rho(\delta_i' \circ \delta_i) = \rho(\delta_i') \circ \rho(\delta_i)$, whether or not $\delta_i' \circ \delta_i$ lies in Δ_i. So ρ extends to an action of the closure $\overline{\Delta}_i$ on the range of f: equivariance on the generators is equivariance on the generated semigroup — the induction behind Lemma 7. Under standard names, f is a *semiconjugacy*, or *factor map*, onto its image, and the descent along f is the *deterministic lumpability* of the level-set partition, also known as *exact aggregation*.

4 Transform-Context

Setting. Transform independence may be partial in the same way: s_i might be transform-independent of some variables and of its own initial value, with the others retained as its context. We partition $\{1,\dots,n\}$ as $\{i\}\cup J\cup K$ as in Section 2, with J the context and K the remaining variables. The variable s_i is *transform-independent of $\boldsymbol{s}_K$ and of its own initial value given context $\boldsymbol{s}_J$*, with respect to f and Δ_i, if the conditions of Section 3 hold when the auxiliary maps are allowed to depend on the context values $\boldsymbol{a}_J$. The family is unchanged: only the maps consult the context.

Transform-map formulation (no structure on T). The variable s_i is transform-independent of $\boldsymbol{s}_K$ and of its own initial value given context $\boldsymbol{s}_J$, with respect to f and Δ_i, iff there exists a map $H^{\Delta}_{i,J}\colon \mathcal{E}_i^J \to T$ (denoted $H^{\Delta}_{i,J}(f(\boldsymbol{a}),\delta_i,\boldsymbol{a}_J)$, the transform written before the context) such that

$$\forall\, \boldsymbol{a}\in S_1\times\cdots\times S_n,\ \ \forall\,\delta_i\in\Delta_i\quad f(\boldsymbol{a}|_{a_i=\delta_i * a_i}) = H^{\Delta}_{i,J}\big(f(\boldsymbol{a}),\,\delta_i,\,\boldsymbol{a}_J\big), \tag{15}$$

where $\mathcal{E}_i^J \coloneqq \{(f(\boldsymbol{a}),\,\boldsymbol{a}_J) : \boldsymbol{a}\in S_1\times\cdots\times S_n\}\times\Delta_i$ — the transform still constrains nothing, being chosen independently of $\boldsymbol{a}$, while the output and the context may constrain each other. Setting $J=\emptyset$ recovers (11); setting $J=\{1,\dots,n\}\setminus\{i\}$ does not hold trivially: the map consults the transform and every remaining coordinate, still not the initial value the transform acts on.

Remark 20 (Transform output-determinacy on context J). For every $\boldsymbol{a},\boldsymbol{b}\in S_1\times\cdots\times S_n$, if $f(\boldsymbol{a})=f(\boldsymbol{b})$ and $\boldsymbol{a}_J=\boldsymbol{b}_J$, then $f(\boldsymbol{a}|_{a_i=\delta_i * a_i}) = f(\boldsymbol{b}|_{b_i=\delta_i * b_i})$ for every $\delta_i\in\Delta_i$.

As in the full case, transform output-determinacy on context J is necessary and sufficient for $H^{\Delta}_{i,J}$ to be well-defined on $\mathcal{E}_i^J$. Setting $J=\emptyset$ recovers Remark 4. Geometrically, this is still the descent of Section 3, now within each context slice: with $\boldsymbol{s}_J$ fixed, applying a transform carries every level set of the partial function into a single level set — in general a different one.

Remark 21 (Partial-function form). For every $\boldsymbol{c}_J\in\prod_{j\in J}S_j$, the variable s_i is fully transform-independent of the remaining variables and of its own initial value on the partial function $f|_{\boldsymbol{s}_J=\boldsymbol{c}_J}\colon \prod_{m\in\{i\}\cup K}S_m\to T$, with respect to the same family Δ_i, in the sense of Section 3.

Remark 21 is equivalent to s_i being transform-independent of $\boldsymbol{s}_K$ and of its own initial value given context $\boldsymbol{s}_J$: since $\boldsymbol{a}|_{a_i=\delta_i * a_i}$ fixes $\boldsymbol{a}_J$, the map $H^{\Delta}_{i,J}(\cdot\,,\cdot\,,\boldsymbol{c}_J)$ is the transform map of $f|_{\boldsymbol{s}_J=\boldsymbol{c}_J}$. Partial transform independence is full transform independence on sets of partial functions, the family shared across them; context is what's fixed.

Finite-difference formulation (abelian T). When $(T,+)$ is an abelian group, the condition has the equivalent algebraic form: there exists $G^{\Delta}_{i,J}\colon \mathcal{E}_i^J\to T$ such that

$$\forall\, \boldsymbol{a}\in S_1\times\cdots\times S_n,\ \ \forall\,\delta_i\in\Delta_i\quad f(\boldsymbol{a}|_{a_i=\delta_i * a_i}) - f(\boldsymbol{a}) = G^{\Delta}_{i,J}\big(f(\boldsymbol{a}),\,\delta_i,\,\boldsymbol{a}_J\big). \tag{16}$$

The relationships between (15) and (16) parallel items (a) and (b) of Section 3, with the context tuple $\boldsymbol{a}_J$ entering each map as a passenger argument: the formulations are related by $G^{\Delta}_{i,J}(v,\delta_i,\boldsymbol{a}_J) = H^{\Delta}_{i,J}(v,\delta_i,\boldsymbol{a}_J) - v$ and $H^{\Delta}_{i,J}(v,\delta_i,\boldsymbol{a}_J) = v + G^{\Delta}_{i,J}(v,\delta_i,\boldsymbol{a}_J)$, and no derivative formulation is included.

The apparatus under context. Every result of Section 3 applies within each context slice through the partial-function form, with $f|_{\boldsymbol{s}_J=\boldsymbol{c}_J}$ in the role of f: each is proved for a function on a total product of factor sets, and each partial function is one. Remark 5 converts the slice's transform map into a transition map on the substitutions the family represents. The closure $\overline{\Delta}_i$ of Remark 6, Lemma 7, and Proposition 8 apply per slice with one and the same closure: the closure and the reach hypothesis consult S_i and Δ_i alone, so a single hypothesis serves every slice at once. Read back through Remark 3, the slice conclusions reassemble into context statements: under the hypothesis of Proposition 8, if s_i is transform-independent of $\boldsymbol{s}_K$ and of its own initial value given context $\boldsymbol{s}_J$ with respect to f and Δ_i, then s_i is transition-independent of $\boldsymbol{s}_K$ given context $\boldsymbol{s}_J$ with respect to f. Corollary 9, Definition 10, and Corollary 11 apply per slice verbatim, achievability being likewise independent of the context. The safe transforms of Remark 12 are read per slice — a transform safe for one partial function need not be safe for another — and Lemma 13, Lemma 14, Proposition 15, Corollary 16, and Remarks 17 and 18 hold in each slice so read. Remark 19 reads likewise, with ρ acting on the range of the partial function.

5 Contextual Order

Setting. The inputs $s_1, \ldots, s_n$ are *contextually ordered* with respect to f iff for every i, the variable s_i is transition-independent of $\boldsymbol{s}_{\{1,\ldots,i-1\}}$ given context $\boldsymbol{s}_{\{i+1,\ldots,n\}}$, in the sense of Section 2: the tuple $(f(\boldsymbol{a}),\, a_i,\, \boldsymbol{a}_{\{i+1,\ldots,n\}})$ is a sufficient representation for predicting how f responds to movement of s_i. The contexts nest and shrink along the order. At $i = n$ the context is empty and the condition is transition independence in the sense of Section 1: s_n is transition-independent of the remaining variables $\boldsymbol{s}_{j\neq n}$ with respect to f. At $i = 1$ the condition holds trivially. The property is of the ordering as given; when some reordering of the inputs is contextually ordered, the inputs are *contextually orderable*.

Remark 22 (Adjacent grouping). Grouping adjacent variables preserves contextual order: partition the indices into consecutive blocks and read f on the product of the block products, each block tuple a single variable — the block tuples are contextually ordered with respect to f so read. Substituting a block from its last variable to its first, each step's context — the remainder of its own block, already substituted, and the later blocks — stands at values shared by any two tuples the substitution must reconcile, so output-determinacy on the context (Remark 2) carries equality of outputs through every step, and the composite response is determined by the output, the block's values, and the later blocks' values alone. In particular, split in two: for $\{s_1, \ldots, s_m\}$ and $\{s_{m+1}, \ldots, s_n\}$, the latter tuple $\boldsymbol{s}_{\{m+1,\ldots,n\}}$ is transition-independent of $\boldsymbol{s}_{\{1,\ldots,m\}}$ with respect to f so read. The preservation is one-way: the grouped order returns each variable's condition only with the other variables of its block as added context, and the unconditional independence of the last variable is exactly what a coarser order does not recover.